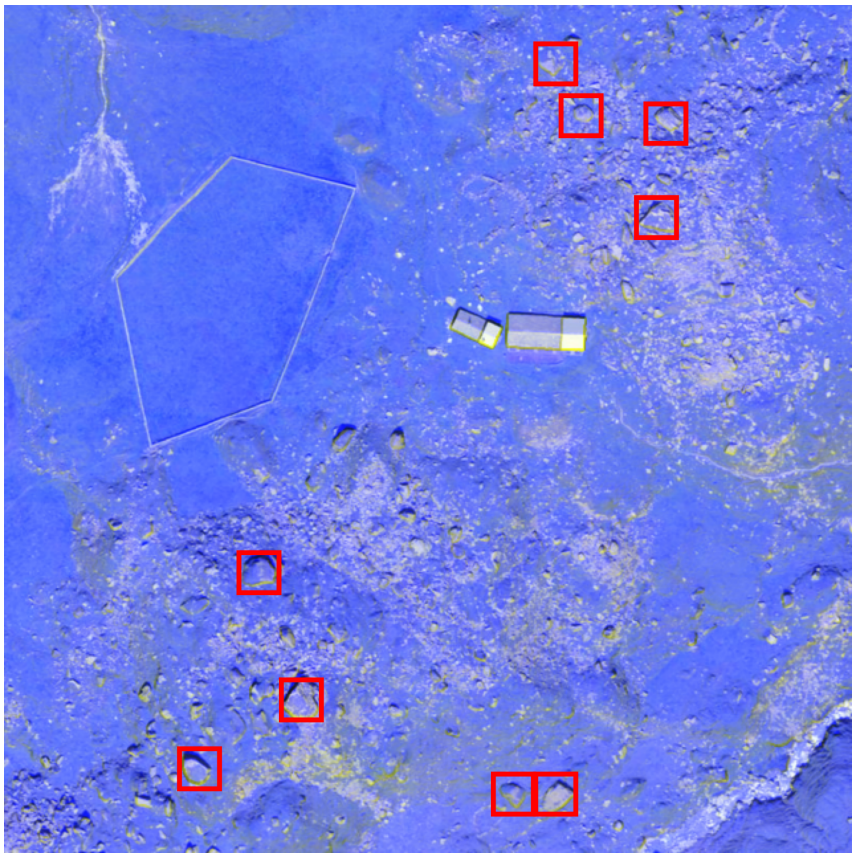
EPFL

ENV-540 January 23, 2026

Large Rock Detection

Image Processing for Earth Observation



Matthias Broch, Theo Houle, Kelan Solomon

Professor: Prof. Devis Tuia
GitHub: https://github.com/kelansolo/Large-Rocks-Detection

Contents

1 Introduction **1**

2 Literature Review **1**

3 Data Processing **1**

3.1 Band Selection 2

3.2 Dataset Tinder 2

3.3 Data Workflow 2

3.3.1 Merging the Bands - `merge_all_bands.py` 3

3.3.2 Splitting into Train, Validation and Test Set - `split.py` 3

3.3.3 Annotation of Large Rocks - `annotate.py` 3

3.3.4 Choosing certain Bands from the Five-Band Images 3

4 Model **3**

4.1 YOLO 3

4.2 Metrics 4

4.3 YOLO Model Selection 4

5 Results **5**

5.1 Baseline 5

5.2 Model with 3 Band Images 5

5.2.1 Training Methodology 5

5.2.2 Discussion 6

5.3 Model with 4 Band Images 7

5.3.1 Training Methodology 7

5.3.2 Discussion 8

6 Comparison & Discussion **8**

7 Final results on test dataset **9**

8 Alternative Solution **9**

9 Conclusion **10**

A Initial Data Analysis

B Model selection

C 3 Band training metrics

D 4 Band training metrics

E Alternative Solution

1 Introduction

Mapping large rocks is essential for topographic maps. Traditionally, swisstopo relies on manual annotations, a labor-intensive process. This project explores automating large rock detection (over 5x5 meters) using high-resolution aerial imagery and digital surface models (DSM). Leveraging deep learning, we aim to develop reliable detection pipelines that balance precision and recall, streamlining swisstopo workflows.

2 Literature Review

With the development of YOLO (You Only Look Once) and its architectural successors, the accuracy of object detection has significantly improved and these models are now considered state of the art [1]. Detecting large rocks in remote sensing data poses challenges like arbitrary object orientations, complex backgrounds, and labour-intensive annotation. Rotation-insensitive models and improved feature extraction address these issues [2]. Integrating DSM data with RGB imagery enhances detection accuracy [3]. However, annotation remains a key bottleneck, requiring extensive expertise and time [2].

3 Data Processing

The dataset is provided by SwissTopo and can be downloaded here [1]. It consists of 4'354 large rock annotations scattered across thousands of satellite images. Each image has a size of 640 by 640 pixels and exists in three versions[2]:

- RGB: images provided by SwissImage (originally 10cm resolution but reduced to 50 to unify across all images)
- SwissSurface3D: images from a LiDAR sensor. These represent digital surface models (DSM)
- SwissSurface3D Hillshade: images that are the DSM from above passed through the hillshade function from QGIS.

With these, a JSON file is provided containing for each image, the position of the rocks as defined by the annotator from SwissTopo.

Finally, the dataset also provides useful tips via a jupyter notebook, guiding our initial research.

[1] https://enacshare.epfl.ch/bY2wS5TcA4CefGks7NtXg

[2] Throughout this report, we will typically refer to these as RGB, DSM and hillshade, respectively.

3.1 Band Selection

RGB comprises 3 bands, while DSM and Hillshade each consist of 1 band, resulting in a total of 5 bands. This naturally led to questions about which bands to include and whether both DSM and Hillshade were necessary, given that Hillshade is derived from DSM.

The tip file provided with the dataset suggested to prefer Hillshade over DSM. For this reason, we decided to discard DSM completely. Furthermore, for reasons explained later in Section 4, we need to have three bands. Thus we thought that we should have a combination of two of the RGB bands and Hillshade. We tested which combination yielded the best results by running the same model on the different mixes of bands and elected to use red, green and hillshade. The result showed that this setup was consistently among the best metrics. See Table 8 and Figure 2 for details.

3.2 Dataset Tinder

During our initial exploration of the data, we discovered that some of the annotations in our dataset would be extremely hard for our model to predict. Two such examples are shown in Figure 3, Appendix A. We decided to go through our data and manually remove the images that we thought would be hard to predict, leaving us with about 700 images (approximately 70% of the original dataset). For this, we wrote a program which showed an image at a time with "good" and "bad" buttons and sorted the image into the corresponding folder.

Throughout this report, we will refer to this filtered version of the dataset as the "good" dataset.

3.3 Data Workflow

Below we describe the steps of our data processing pipeline to produce from the original dataset (or the "good" dataset) a dataset ready to be used by our model (see section 4). Figure 1 gives a visual representation of the data processing pipeline. The different steps are detailed below.

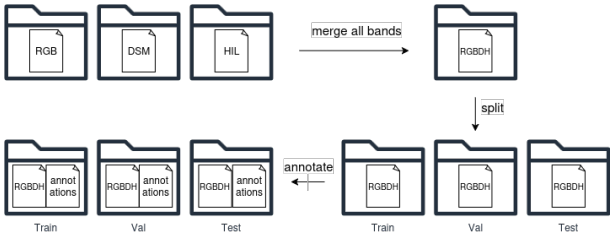


Figure 1: Flowchart of the data pipeline

3.3.1 Merging the Bands - `merge_all_bands.py`

The script combines the RGB, DSM, and Hillshade bands into a unified multi-channel format. These layers are concatenated along the channel axis, resulting in a five-channel image that integrates both visual and elevation information. The output format for these merged images is TIFF, which ensures compatibility and consistency for subsequent processing tasks.

3.3.2 Splitting into Train, Validation and Test Set - `split.py`

This script splits a dataset into training, validation, and test subsets based on user-defined fractions (default: 60% training, 20% validation, 20% testing). Users can specify the total test set size. The script shuffles the images, calculates subset sizes, and copies files into corresponding output directories, preparing the data for model training and evaluation.

3.3.3 Annotation of Large Rocks - `annotate.py`

This script generates annotation files for images based on a JSON file with label data, following COCO dataset format guidelines. It verifies the input directory and JSON file, retrieves rock annotations for each image, and creates bounding boxes (10 pixels, 5x5 meters) around annotated points. Each annotation file includes the class label, normalized rock coordinates, and bounding box dimensions, saved as .txt files alongside images. In our case, there is only a single class, namely `gros_cailloux` (id 0).

3.3.4 Choosing certain Bands from the Five-Band Images

We have now reached the final step of the preprocessing pipeline, illustrated in Figure 1 (bottom left). This script creates a copy of the dataset, retaining only the specified bands (ranging from a single band to all bands) and a chosen fraction of the dataset. This flexibility enables testing different band combinations on any subset of the prepared data, making it a valuable tool for quick testing.

4 Model

4.1 YOLO

For this project, we chose to use YOLO[3], a decision strongly influenced by the recommendation of the authors of the dataset. After careful consideration, we determined that following their suggestion was the most logical approach. YOLO is widely recognized as a state-of-the-art model for object detection and has consistently demonstrated good

[3]https://docs.ultralytics.com/

performance in tasks similar to ours. Moreover, its extensive use in the literature we reviewed [1] further reinforced our confidence in its suitability. By leveraging a tool that is both highly effective and well-documented, we aimed to maximize the project's chances of success while aligning with established best practices in the field.

4.2 Metrics

Our primary metrics for comparing models are precision, recall, and mAP50 (mean average precision at a 50% intersection over the union threshold). These metrics are straightforward to interpret at a glance and are automatically output by YOLO during training. Since our goal is to balance precision and recall—recognizing that "too many false positives create lengthy correction tasks for annotators while missing too many targets renders predictions ineffective", we also evaluate performance using the F1 score, which provides a harmonic mean of precision and recall.

4.3 YOLO Model Selection

Several versions of YOLO exist, with the dataset authors recommending version 8. However, newer versions up to 11 have since been released. To explore whether newer versions might yield better results, we wanted to compare versions 8 and 11.

Each version includes models of various sizes, ranging from nano to XL, reflecting their number of parameters. Given our computational resources, we selected three sizes for testing: nano, large, and extra-large. Nano was included to assess if a lightweight model could match the performance of larger models, potentially reducing deployment complexity. Extra-large leveraged our ability to handle high computational loads, while large provided a middle-ground tradeoff between size and complexity.

Table 9 presents the training results for all tested model variations, while Figure 4 provides a visual comparison across the different metrics. The results indicate that, in nearly all cases, the version 8 models perform at least as well as their version 11 counterparts within the same size category. Based on these findings, we chose to proceed with version 8. Additionally, the performance of the large and extra-large models in version 8 is comparable, so we plan to use either of them for the remainder of our research.

Finally, it is worth noting that since our choice of bands replaces the blue channel with Hillshade, the images no longer have natural colours. For this reason, we will prefer training models from scratch rather than using pre-trained models trained on RGB images.

5 Results

5.1 Baseline

To establish a baseline for comparison, we trained a basic model on unmodified data to quantify improvements in performance. Specifically, we used a YOLOv8l model trained on the original RGB dataset for 150[4] epochs, with default hyperparameters and no data augmentation. As mentioned before[5], we won't be using a pre-trained model since we don't use RGB images and thus we will train a second baseline without pre-trained weights. This way we can have a more meaningful comparison between models without pre-trained weights while still being able to compare to the best version of the baseline. The validation metrics from this training are summarised in Table 1.

	mAP50	mAP50-95	mAP75	F1 Score	Precision	Recall
scratch	0.13	0.034	0.0063	0.23	0.23	0.23
pre-trained	0.18	0.056	0.014	0.30	0.30	0.29

Table 1: Validation metrics for 400 epochs of the baseline model and dataset

Our research will be to try and find some way to improve results of the baseline established by exploiting the extra bands given and tuning the model.

5.2 Model with 3 Band Images

5.2.1 Training Methodology

Following the results presented in Section 3 and 4 concerning the best dataset and best model to use, the first thing we tried is training YOLOv8l from scratch on 3 bands images using red, green and hillshade for 150 epochs (to match the baseline). Figure 5 shows the metric plots after 150 epochs. We can see that the precision, recall and mAPs have not yet plateaued, additionally, the loss curves (particularly val/box_loss and val/cls_loss) haven't completely stabilised, indicating that we could benefit from more epochs. Thus we resumed the training for 250 more epochs and got the metric plots shown in Figure 6. Since these plot still didn't seem to show a clear plateau, we ran again for 800 epochs (see figure 7) and now, these plots definitely show a plateau starting to form.

While training for 800 epochs yields better results, overfitting is a concern. To alleviate this concern, we use the validation set to ensure generalizability. The validation metrics at each epoch steps are summarised in table 2.

While these results are not too bad compared to the baseline, we believe there is still room for improvement. At this stage, we decided to experiment with larger bounding

[4]150 was the number of epochs suggested by the teaching assistants

[5]see the last paragraph of section 4.3

	mAP50	mAP50-95	mAP75	F1 Score	Precision	Recall
150 epochs	0.18	0.049	0.010	0.29	0.24	0.36
400 epochs	0.23	0.06	0.011	0.34	0.35	0.33
800 epochs	0.41	0.15	0.05	0.51	0.45	0.52

Table 2: Validation metrics for 400 and 800 epochs

box sizes. Initially, we used 10x10 pixel bounding boxes, corresponding to an area of 5x5 meters, given the 50 cm resolution of the imagery. This choice was motivated by our goal of identifying rocks that are 5x5 meters in size. However, since we also want to capture rocks larger than this, we need bigger bounding boxes as the larger rocks will overspill the 10x10 one.

To address this, we trained a batch for 800 epochs on YOLOv8l, using bounding box sizes of 20x20 and 30x30 pixels. The results of this experiment are summarised in Table 3.

	mAP50	mAP50-95	mAP75	F1 Score	Precision	Recall
20x20	0.57	0.12	0.06	0.62	0.68	0.57
30x30	0.68	0.26	0.048	0.71	0.77	0.66

Table 3: Validation metrics for 800 epochs with bounding boxes of sizes 20x20 and 30x30 pixels

These results represent a significant improvement over the original 10x10 pixel bounding boxes. This improvement can be attributed to the fact that the larger bounding boxes are more likely to encompass all large rocks accurately. While the predictions may now align more easily with the labels due to the increased bounding box size, this is not necessarily a drawback.

We believe this trade-off is acceptable because, even if a prediction is not perfectly aligned with the rock or the bounding box is slightly oversized, a human interpreting these results could easily make the necessary adjustments.

However, we need to ensure the bounding boxes are not too large otherwise, any wrong prediction from the model that happens to be within a huge bounding box label but not on the rock would still be rewarded. For this reason, we stick with bounding boxes no larger than 30x30 pixels (i.e. 15 meters wide).

5.2.2 Discussion

It turns out that the RGHil band mix performs better than the baseline trained from scratch and is as good as the pre-trained one. This suggests that swapping the blue channel with hillshade does provide improvements. This is not a surprise since hillsahde provides depth information that the model can use to detect the rocks more easily. On top of that,

increasing the number of epochs and the size of the bounding boxes yields better resultsr. The confusion matrix of the best model (800 epochs on YOLOv8l with bounding box size 30x30) is displayed in Figure 8 Appendix C.

5.3 Model with 4 Band Images

Unfortunately, YOLO's default architecture only supports 3-band images. While it's possible to modify the source code to handle 4 bands [4], it requires multiple changes to the Ultralytics library, including restructuring the initial network layer, revising OpenCV's image handling functions, and modifying the plotting functions. Since YOLO relies on OpenCV, which is limited to 4-band images, extending the model to process all 5 bands would require extensive modifications beyond this project's scope. Therefore, we decided to keep RGB and Hillshade.

5.3.1 Training Methodology

We followed a similar methodology as the 3 bands models and trained for 150, 400 and 1000 epochs. The main difference is that we decided to use the X version of YOLOv8 which has more weights. The reason behind this is that we use 4 instead of 3 bands (i.e. more information) and thus we could benefit from more weights. As a result, we decided to increase from 800 to 1000 epochs ensuring enough time for the model to plateau. The training results at every checkpoint can be seen in Figure 9, Figure 10 and Figure 11 respectively in Appendix D

As before, we validated our results with the validation set to ensure generalizability. The training curves reveal a clear plateau across all metrics except for the training box loss, suggesting the model has reached convergence. The performance metrics using the best weights for each model configuration are presented in Table 4.

	mAP50	mAP50-95	mAP75	F1 Score	Precision	Recall
150 epochs	0.20	0.057	0.018	0.30	0.28	0.32
400 epochs	0.28	0.079	0.009	0.40	0.41	0.38
1000 epochs	0.42	0.140	0.025	0.53	0.57	0.49

Table 4: Validation metrics for 150, 400 and 1000 epochs

We evaluated different confidence and intersection-over-union (IoU) thresholds during validation to determine their optimal values. These two hyperparameters can be adjusted post-training to fine-tune the model's performance. Table 10 in Appendix D shows all the metrics at different IoUs and confidence thresholds. Since we would like to balance the precision and recall, we can choose the hyperparameters based on the F1 score. Here that would mean `conf = 0.3` and `iou = 0.5` leaving us with an **F1 score of 52.89%, precision of 56.86% and recall of 49.43%.**

We then ran the model with selected data augmentation techniques, being careful to only implement augmentations that would preserve the shape and size of our images, since rock size is a critical feature for detection. The results for this model are shown in Table 12, Appendix D. We can observe that with the augmentation, the model performs slightly worse, with a max F1 score of just under 50%, this is an unexpected behaviour since we were quite conservative with our augmentations (Listing 1, Appendix D). Perhaps introducing rotational invariance added additional complexity to the problem. Other augmentation configurations were tried, such as changing the HSVs and reducing the rotation but none performed better.

Next, we tried increasing the batch size. We trained a model with a batch size of 64 and tested it on the validation set with varying IoUs and confidence thresholds (Table 11, Appendix D). We observe that while precision has drastically improved, reaching close to **80%**, the recall remains low at only **14%**. Our hypothesis is that due to the inherent complexity of rock detection, the model is overly conservative and only makes predictions when it has very high confidence.

Finally, as with the 3 Band model, we increased the bounding box size to 30x30 pixels, resulting in an **F1 of 73%, a recall of 72% and a precision of 74%.**

5.3.2 Discussion

Since the balance between precision and recall is important, we can conclude that our best option is the model with a batch size of 10 and no data augmentation. With an F1 score of just under 73%, the model is not reliable enough to replace hand annotations. We can see in the confusion matrix in Figure 12 Appendix D that there is still a high number of false positives and false negatives.

One could argue, that we could leverage the high-precision model even if it comes with lower recall. This approach would ensure that annotators could trust the model's predictions, eliminating the need to verify each detection. Instead, annotators could focus exclusively on identifying missed objects. To address the precision-recall imbalance, we could make the detection task more lenient by reducing both the confidence threshold and the IoU requirements, which should increase recall while accepting a reasonable trade-off in precision.

6 Comparison & Discussion

We can see that the 4-band model performs slightly better than the 3-band model, with higher mAP and F1 scores, showing better accuracy and balance between precision and recall. The 4-band model achieves a mAP50 of 0.75 and F1 of 0.73, compared to 0.68 and 0.71 for the 3-band model. Both outperform the baseline models, where the pre-trained version reaches only 0.12 mAP50 and 0.47 F1. This highlights the benefit of adding hillshade data, which improves rock detection by providing useful elevation information. The 4-band model takes significantly more time to train compared to the 3-bands, (4.2

Model	Train [H]	mAP 50	mAP 50-95	mAP 75	F1	Precision	Recall
baseline scratch	1.63	0.043	0.16	0.012	0.27	0.25	0.30
baseline pre-trained	1.63	0.12	0.36	0.030	0.47	0.57	0.41
3 Band	2.32	0.68	0.26	0.048	0.71	0.77	0.66
4 Band	4.22	0.75	0.42	0.43	0.73	0.74	0.72

Table 5: Final performance metrics on the validation dataset

hours versus 2.3 hours respectively) for very little increase in performance. We chose to keep the 4-band model as training time is not our primary concern, but the 3-band model could be a good alternative if we didn't have access to the SCITAS HPC cluster.

7 Final results on test dataset

To assess the model's real-world performance, we evaluated our final configuration—the 4-band model trained with 30x30 bounding boxes—on the held-out test dataset. This evaluation on previously unseen data validates that our model generalizes well beyond the validation set and performs as expected in production conditions.

	mAP50	mAP50-95	mAP75	F1	Precision	Recall
Validation	0.75	0.42	0.43	0.73	0.74	0.72
Test	0.64	0.30	0.20	0.65	0.68	0.63

Table 6: Final performance metrics on test dataset

We can see that the performance metrics are slightly worse on the test set than on the validation set. This performance gap suggests some degree of overfitting, though the model still maintains reasonable detection capabilities.

8 Alternative Solution

Rather than tackling the challenging task of object detection for large rocks, we propose pivoting to image classification as a more effective approach. Since a significant portion of our dataset consists of rock-free images (close to 50%!), a classifier could identify these cases, substantially reducing the annotation workload. By focusing on obtaining a high recall, we could be reasonably confident that the images that are deemed rock-free are, in fact, free of rocks. This classification step streamlines the annotation process by allowing annotators to skip images that have been confidently classified as containing no rocks. It could also strengthen our rock-detection pipeline by acting as an initial filter, preventing false positives in rock-free images and improving the system's overall accuracy. This simpler

classification approach addresses the core challenge more efficiently than attempting to solve the more complex object detection problem directly.

Moreover, this alternative could also be used as a simplified way to extend the initial dataset. Someone labelling by hand the satellite images could benefit from this option as they would have to label fewer images (assuming the number of rock and rock-free images remains about 50/50).

To evaluate the feasibility of this alternative approach, we conducted experimental evaluations using three distinct classification models. We started with a simple PyTorch CNN, illustrated in Listing 2, Appendix E. We then explored YOLO classification and ResNet50 (Listing 3, Appendix E). To compare the models, we used the F2-score, a weighted harmonic mean of precision and recall that gives twice as much weight to recall compared to the standard F1-score.

Model	Recall	Precision	F2 Score
PyTorch CNN	0.44	0.41	0.43
ResNet50	0.36	0.42	0.37
YOLO Classification	0.90	0.84	0.89

Table 7: Classification model metrics on test dataset

The Yolo classifier substantially outperforms the two other options with an F2 score of **89%** and is a viable solution to our problem. This classifier could be further improved by adding the 4 bands and exploring batch sizes and data augmentation. We could also envisage modifying the confidence thresholds to further boost the recall.

While these are only basic experiments, they demonstrate the feasibility of this alternative. We believe that this is a viable alternative that could be rapidly implemented and improved with very little effort.

9 Conclusion

In this project, we trained and evaluated multiple models to detect large rocks in satellite images, optimizing their performance through adjustments to hyperparameters and data configurations. While our final model, utilizing a 4-band setup and enlarged bounding boxes, achieved a commendable F1 score of 73% on the validation dataset, the results indicate that further refinement is necessary to meet the demands of automated annotation.

To address current limitations, we explored an alternative approach focused on binary classification, demonstrating its potential to streamline the annotation process by effectively filtering rock-free images, with our final model achieving an F2 score of 89%. This complementary strategy, combined with further development of the detection model, provides a promising pathway to reduce annotator workload while maintaining accuracy.

A Initial Data Analysis

Model	Train [H]	mAP50-95	mAP50	mAP75	F1	Precision	Recall
rgb	1.635	0.048525	0.16804	0.01589	0.27110	0.30155	0.24624
rgh	1.639	0.061286	0.21288	0.01547	0.32723	0.36024	0.29977
rgd	1.625	0.011675	0.04729	0.00154	0.08835	0.07199	0.11434
rbd	1.635	0.034491	0.12647	0.00763	0.21481	0.23817	0.19563
rbh	1.635	0.048941	0.16784	0.01823	0.27775	0.25714	0.30194
gbd	1.630	0.027752	0.09992	0.00720	0.11828	0.06838	0.43770
gbh	1.634	0.048198	0.17380	0.01045	0.27699	0.26943	0.28500

Table 8: Metrics resulting from 400 epochs training with YOLOv8l on the good dataset with different mixtures of bands

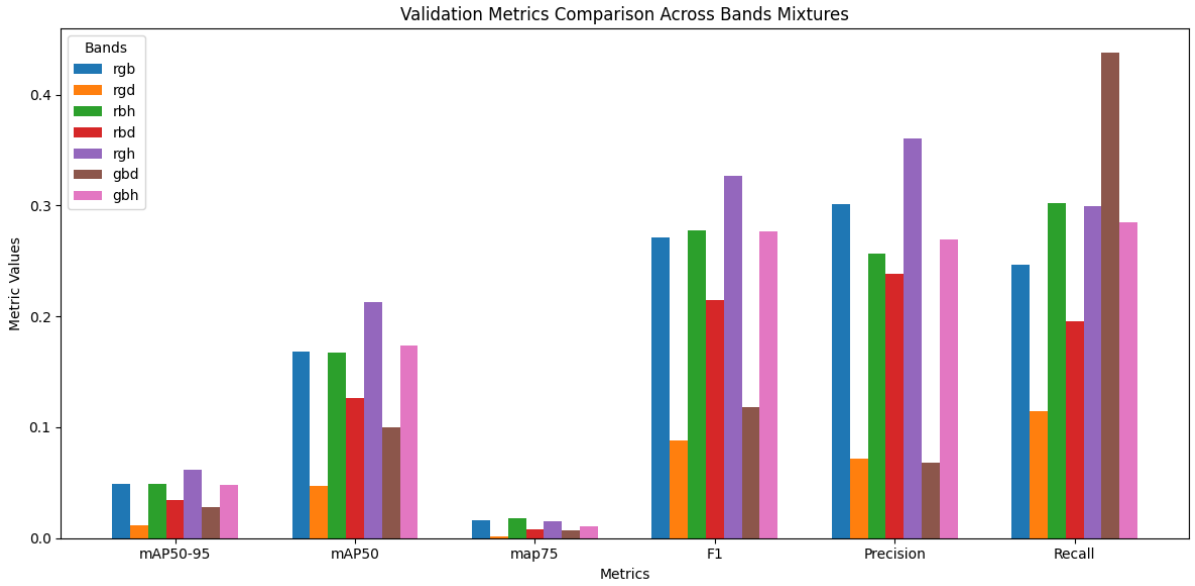


Figure 2: Performance metrics comparison across band mixtures after training for 400 epochs on YOLOv8l on the good dataset

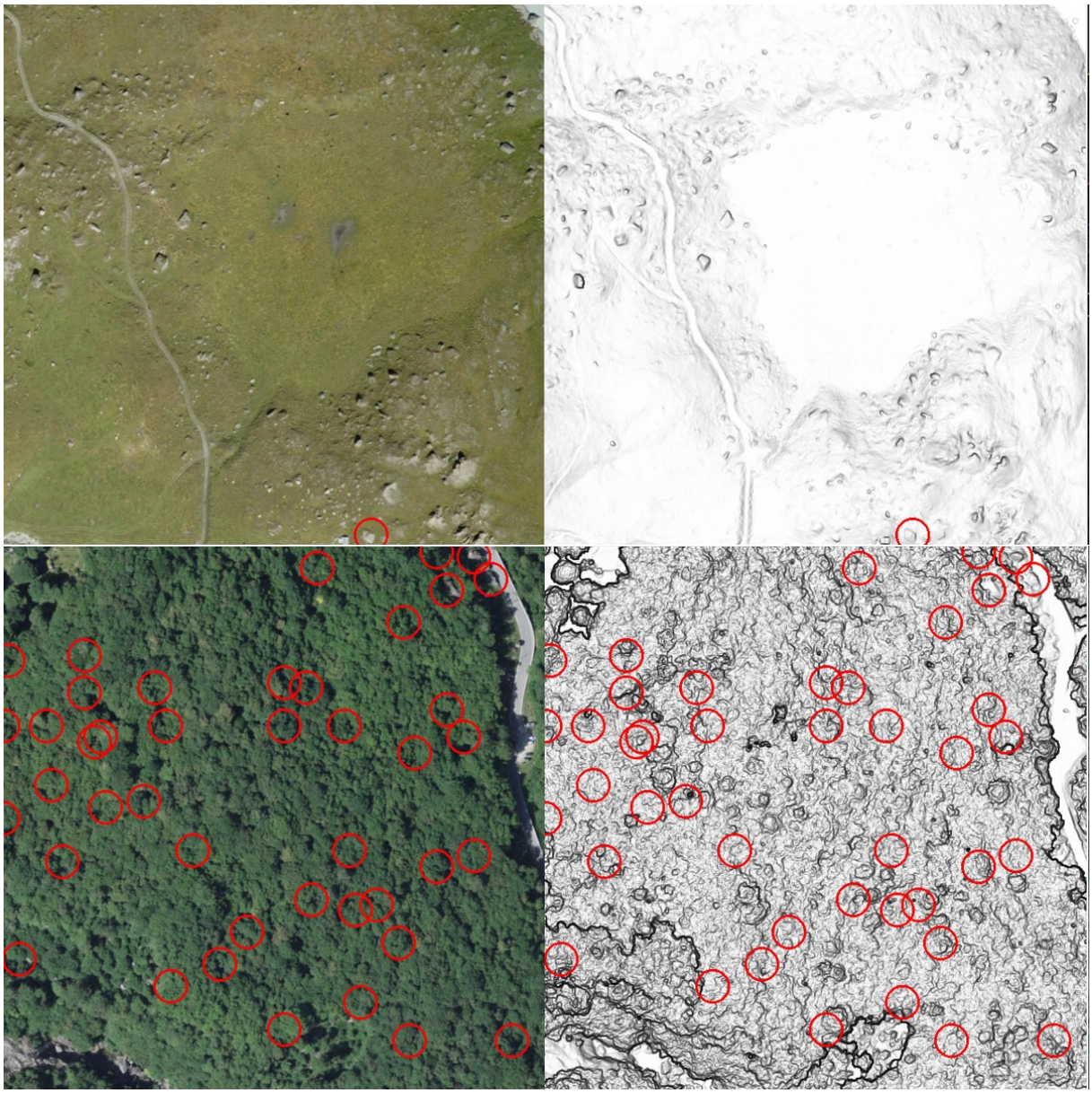


Figure 3: Examples of poor annotations: The top image shows missed rocks, with one circled at the bottom but several overlooked above. The bottom image has circles placed on forested areas where no rocks are visible.

B Model selection

Model	Train [H]	mAP50-95	mAP50	mAP75	F1	Precision	Recall
Yolov8n	0.468	0.09166	0.31689	0.02784	0.41694	0.44521	0.39205
Yolov8l	1.012	0.155068	0.46904	0.06380	0.53406	0.53689	0.53125
Yolov8x	1.514	0.157570	0.40487	0.05241	0.51230	0.51041	0.51420
Yolo11n	0.498	0.08510	0.26845	0.03232	0.34254	0.32629	0.36049
Yolo11l	1.025	0.134123	0.40138	0.05630	0.48639	0.48698	0.48580
Yolo11x	1.516	0.120981	0.35345	0.04557	0.48410	0.49113	0.47727

Table 9: Metrics resulting from 400 epochs training on the RGHil good dataset for all relevant models

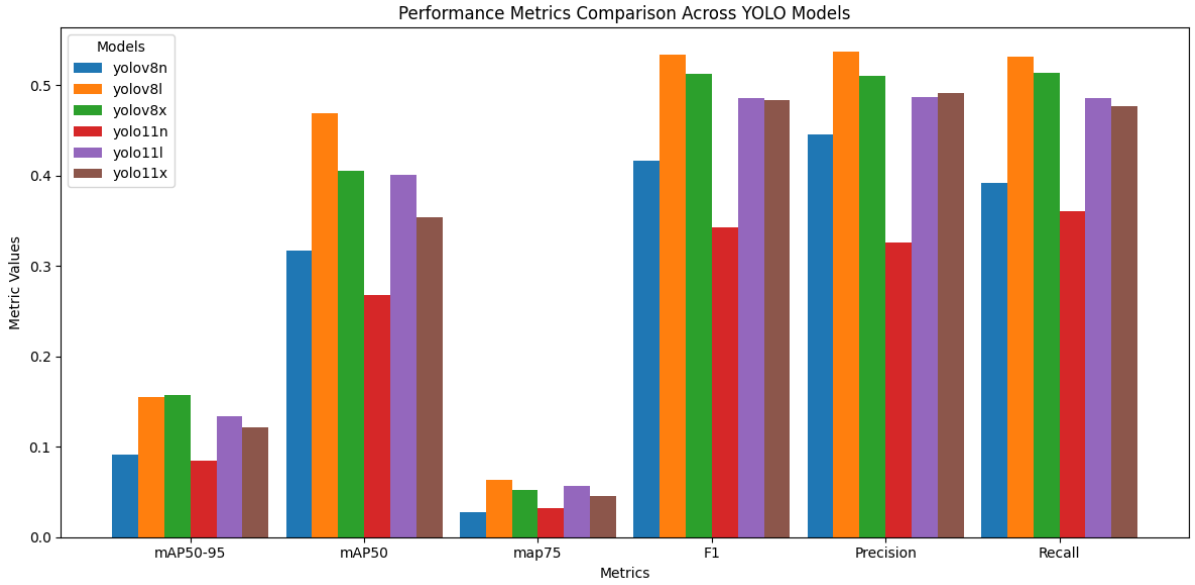


Figure 4: Performance metrics comparison across YOLO models after training for 400 epochs on RGHil images on the good dataset

C 3 Band training metrics

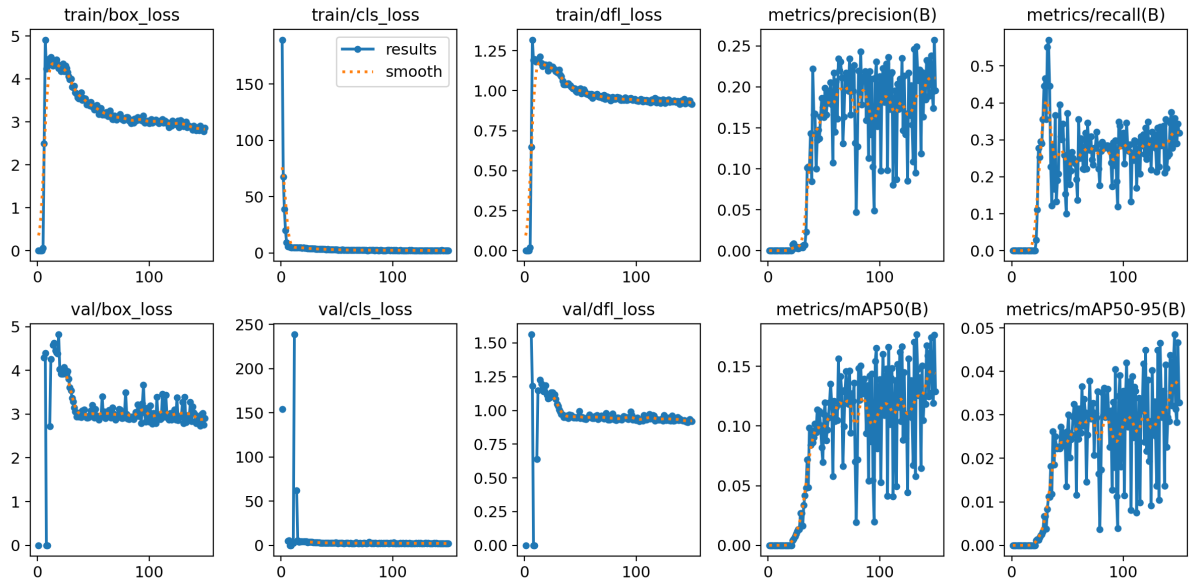


Figure 5: Training metrics after 150 epochs for 3 bands RGHil on YOLOv8l

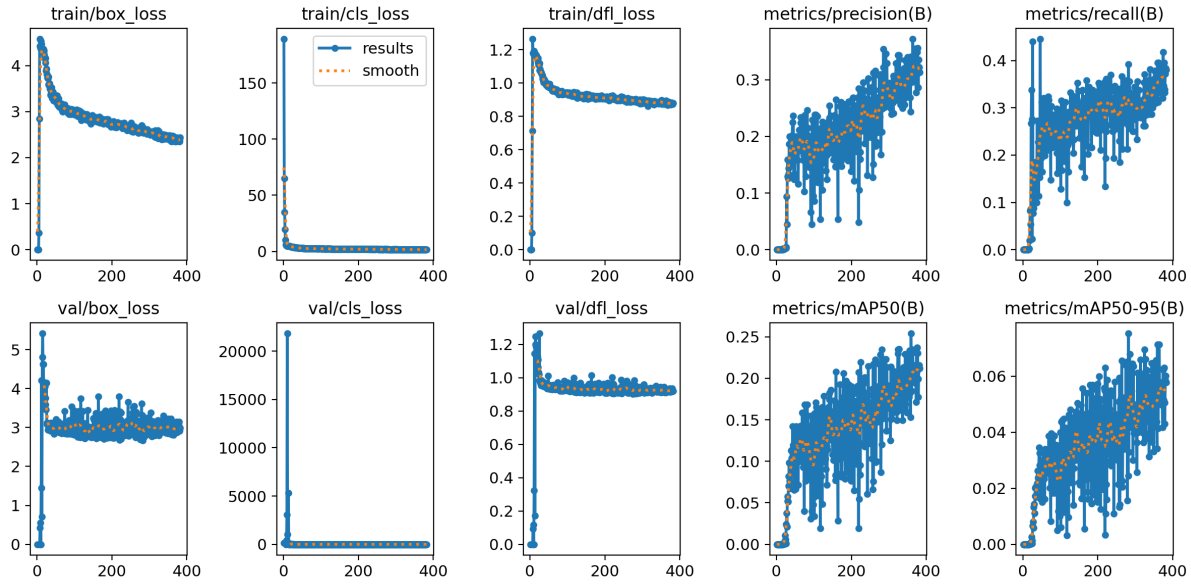


Figure 6: Training metrics after 400 epochs for 3 bands RGHil on YOLOv8l

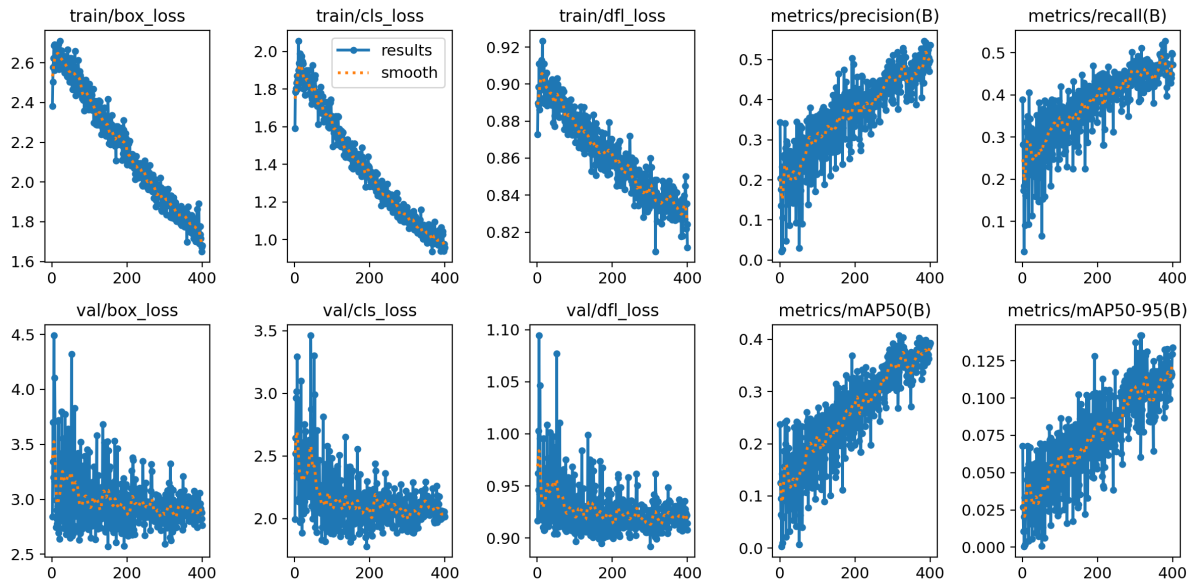


Figure 7: Training metrics after 800 epochs for 3 bands RGHil on YOLOv8l

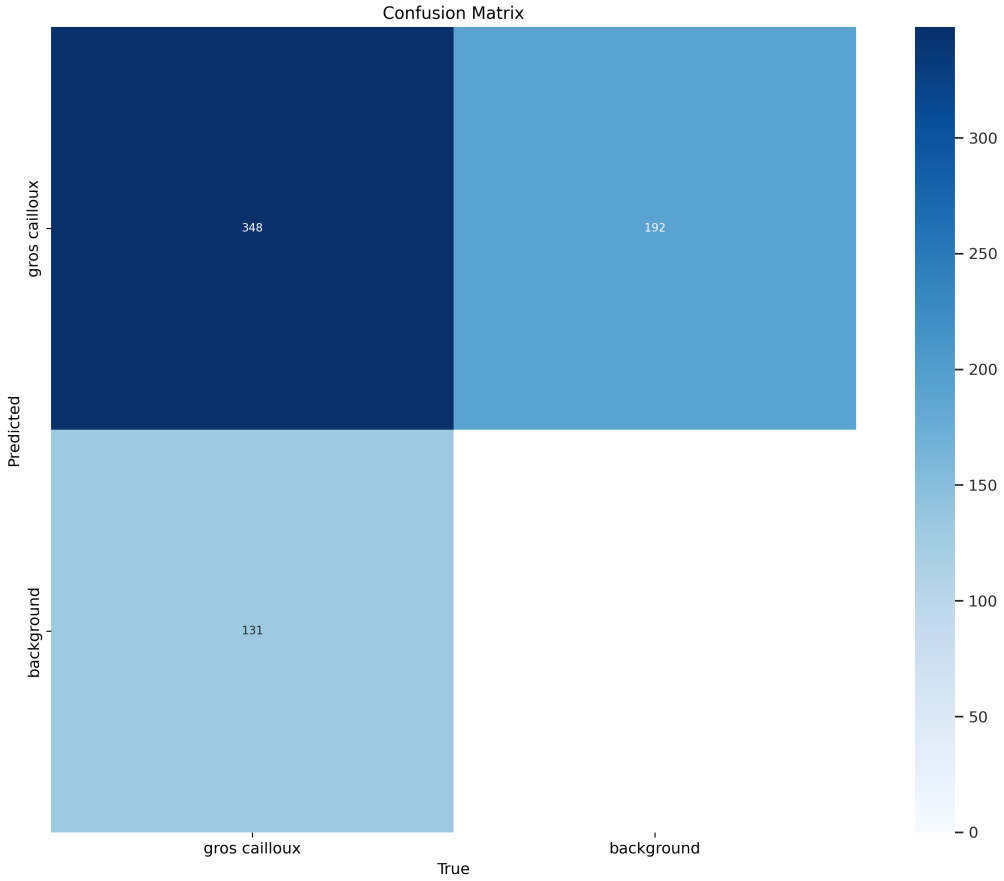


Figure 8: Confusion matrix for final 3 band model using YOLOv8l on RGHil with bbox size 30x30

D 4 Band training metrics

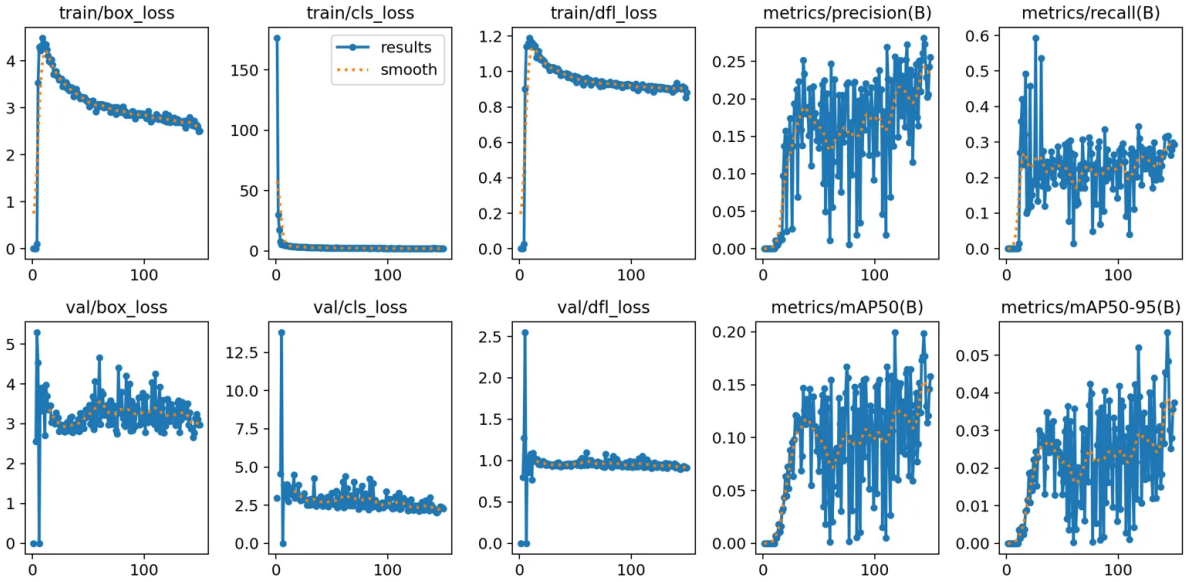


Figure 9: Training metrics after 150 epochs for 4 bands RGBHil on YOLOv8x.

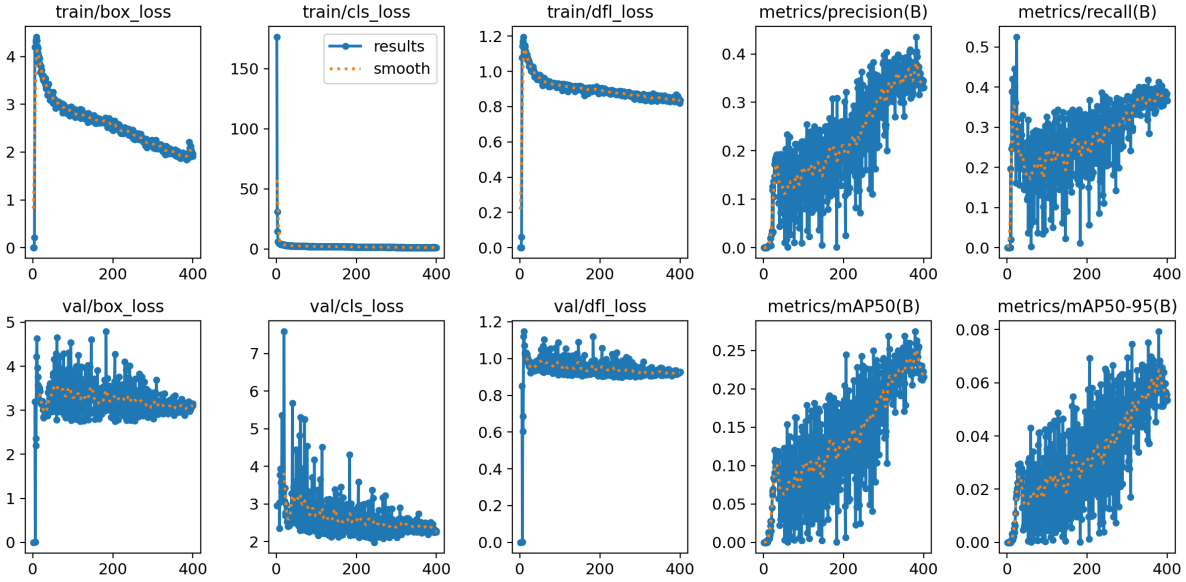


Figure 10: Training metrics after 400 epochs for 4 bands RGBHil on YOLOv8x

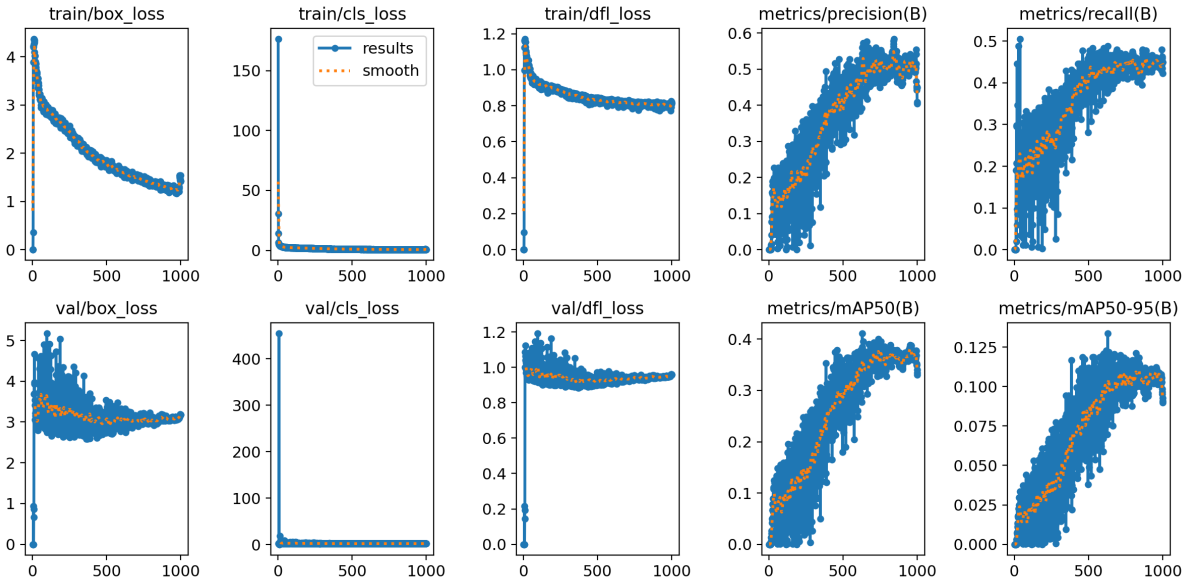


Figure 11: Training metrics after 1000 epochs for 4 bands RGBHil on YOLOv8x

conf	iou	map50-95	map50	map75	f1	p	r
0.1	0.5	0.1658	0.4627	0.0444	0.5289	0.5686	0.4943
0.1	0.6	0.1657	0.4618	0.0448	0.5289	0.5686	0.4943
0.1	0.7	0.1650	0.4576	0.0452	0.5263	0.5663	0.4915
0.1	0.8	0.1620	0.4475	0.0452	0.5244	0.5619	0.4915
0.1	0.9	0.1593	0.4389	0.0459	0.5194	0.5521	0.4903
0.2	0.5	0.1728	0.4731	0.0488	0.5289	0.5686	0.4943
0.2	0.6	0.1729	0.4728	0.0487	0.5289	0.5686	0.4943
0.2	0.7	0.1726	0.4709	0.0489	0.5263	0.5663	0.4915
0.2	0.8	0.1698	0.4635	0.0479	0.5244	0.5619	0.4915
0.2	0.9	0.1669	0.4550	0.0467	0.5194	0.5521	0.4903
0.3	0.5	0.1815	0.4891	0.0546	0.5289	0.5686	0.4943
0.3	0.6	0.1815	0.4891	0.0546	0.5289	0.5686	0.4943
0.3	0.7	0.1814	0.4872	0.0554	0.5263	0.5663	0.4915
0.3	0.8	0.1804	0.4840	0.0545	0.5244	0.5619	0.4915
0.3	0.9	0.1784	0.4772	0.0541	0.5194	0.5521	0.4903
0.4	0.5	0.1879	0.4936	0.0620	0.5257	0.5808	0.4801
0.4	0.6	0.1879	0.4936	0.0620	0.5257	0.5808	0.4801
0.4	0.7	0.1879	0.4936	0.0620	0.5257	0.5808	0.4801
0.4	0.8	0.1879	0.4925	0.0616	0.5240	0.5768	0.4801
0.4	0.9	0.1873	0.4896	0.0609	0.5200	0.5671	0.4801

Table 10: Performance metrics varying confidence and IoU thresholds. Batch size = 10.

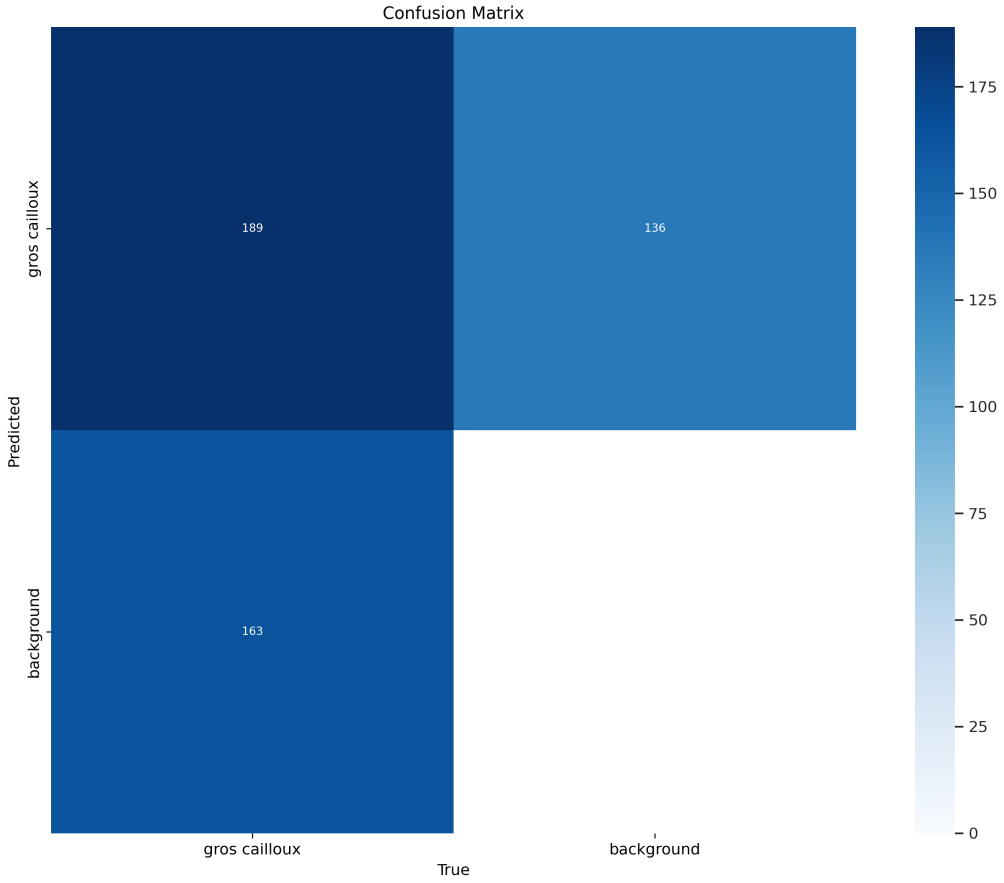


Figure 12: Confusion matrix for final 4 band model using YOLOv8x on RGBHil

conf	iou	map50-95	map50	map75	f1	p	r
0.1	0.5	0.2374	0.4861	0.1939	0.3840	0.7459	0.2585
0.1	0.6	0.2374	0.4861	0.1939	0.3840	0.7459	0.2585
0.1	0.7	0.2374	0.4861	0.1939	0.3840	0.7459	0.2585
0.1	0.8	0.2374	0.4861	0.1939	0.3840	0.7459	0.2585
0.1	0.9	0.2266	0.4633	0.1867	0.3776	0.7000	0.2585
0.2	0.5	0.2301	0.4683	0.1833	0.3282	0.7475	0.2102
0.2	0.6	0.2301	0.4683	0.1833	0.3282	0.7475	0.2102
0.2	0.7	0.2301	0.4683	0.1833	0.3282	0.7475	0.2102
0.2	0.8	0.2301	0.4683	0.1833	0.3282	0.7475	0.2102
0.2	0.9	0.2212	0.4504	0.1751	0.3246	0.7115	0.2102
0.3	0.5	0.2264	0.4668	0.1705	0.2943	0.7711	0.1818
0.3	0.6	0.2264	0.4668	0.1705	0.2943	0.7711	0.1818
0.3	0.7	0.2264	0.4668	0.1705	0.2943	0.7711	0.1818
0.3	0.8	0.2264	0.4668	0.1705	0.2943	0.7711	0.1818
0.3	0.9	0.2181	0.4496	0.1634	0.2916	0.7356	0.1818
0.4	0.5	0.2251	0.4605	0.1775	0.2410	0.7937	0.1421
0.4	0.6	0.2251	0.4605	0.1775	0.2410	0.7937	0.1421
0.4	0.7	0.2251	0.4605	0.1775	0.2410	0.7937	0.1421
0.4	0.8	0.2251	0.4605	0.1775	0.2410	0.7937	0.1421
0.4	0.9	0.2195	0.4483	0.1723	0.2398	0.7692	0.1421

Table 11: Performance metrics varying confidence and IoU thresholds. Batch size = 64.

conf	iou	map50-95	map50	map75	f1	p	r
0.1	0.5	0.1504	0.4252	0.0492	0.4931	0.5181	0.4704
0.1	0.6	0.1494	0.4221	0.0493	0.4949	0.5241	0.4688
0.1	0.7	0.1489	0.4239	0.0498	0.4950	0.5243	0.4688
0.1	0.8	0.1452	0.4135	0.0478	0.4923	0.5183	0.4688
0.1	0.9	0.1430	0.4056	0.0491	0.4873	0.5074	0.4688
0.2	0.5	0.1590	0.4422	0.0561	0.4931	0.5181	0.4704
0.2	0.6	0.1585	0.4404	0.0567	0.4949	0.5241	0.4688
0.2	0.7	0.1585	0.4427	0.0582	0.4950	0.5243	0.4688
0.2	0.8	0.1551	0.4336	0.0562	0.4923	0.5183	0.4688
0.2	0.9	0.1523	0.4258	0.0558	0.4873	0.5074	0.4688
0.3	0.5	0.1627	0.4506	0.0575	0.4931	0.5181	0.4704
0.3	0.6	0.1621	0.4493	0.0571	0.4949	0.5241	0.4688
0.3	0.7	0.1620	0.4491	0.0579	0.4950	0.5243	0.4688
0.3	0.8	0.1601	0.4432	0.0573	0.4923	0.5183	0.4688
0.3	0.9	0.1579	0.4366	0.0567	0.4873	0.5074	0.4688
0.4	0.5	0.1681	0.4581	0.0653	0.4962	0.5272	0.4688
0.4	0.6	0.1679	0.4577	0.0652	0.4955	0.5255	0.4688
0.4	0.7	0.1679	0.4577	0.0652	0.4955	0.5255	0.4688
0.4	0.8	0.1670	0.4557	0.0646	0.4925	0.5189	0.4688
0.4	0.9	0.1656	0.4510	0.0649	0.4860	0.5046	0.4688

Table 12: Performance metrics varying confidence and IoU thresholds. With Data augmentations and Batch size = 10

```yaml
augment:
    hsv_h: 0.00 # Set to 0 to avoid altering colours.
    hsv_s: 0.0 # Set to 0 to avoid altering colours.
    hsv_v: 0.0 # Set to 0 to avoid altering colours.
    degrees: 360.0
    translate: 0.2 # 0.2 is commonly used and works well.
    scale: 0.0 # Disabled to maintain consistent image size.
    shear: 0.0 # Disabled to avoid altering object proportions.
    perspective: 0.0 # Disabled to avoid altering object proportions.
    flipud: 0.5 # Probability of flipping the image vertically.
    fliplr: 0.5 # Probability of flipping the image horizontally.
    mosaic: 1.0 # Probability of applying a mosaic augmentation
    mixup: 0.0
    copy_paste: 0.0
```

Listing 1: Data augmentation hyperparameters

E Alternative Solution

```
class RockDetectionCNN(nn.Module):
    def __init__(self):
        super(RockDetectionCNN, self).__init__()

        # Convolutional layers
        self.features = nn.Sequential(
            nn.Conv2d(4, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout2d(0.3)
        )

        # Adaptive pooling to handle variable input sizes
        self.adaptive_pool = nn.AdaptiveAvgPool2d((4, 4))

        # Fully connected layers
        self.classifier = nn.Sequential(
            nn.Linear(128 * 4 * 4, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.features(x)
        x = self.adaptive_pool(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

Listing 2: Custom PyTorch CNN for classification.

```

class ResNet(nn.Module):
    def __init__(self, input_channel_dim: int = 4, num_classes: int = 1)
    :
        super(ResNet, self).__init__()
        self.model = models.resnet50(num_classes=num_classes)
        self.model.conv1 = torch.nn.Conv2d(input_channel_dim, 64,
    kernel_size=(7, 7),
                                           stride=(2, 2), padding=(3, 3),
    bias=False)
        self.sigmoid = nn.Sigmoid()
        nn.init.kaiming_normal_(self.model.conv1.weight, mode='fan_out',
     nonlinearity='relu')

    def forward(self, x):
        x = self.model(x)
        x = self.sigmoid(x)
```

```
14          return x.squeeze(1)
```

Listing 3: ResNet50 for classification.

References

[1] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using yolo: challenges, architectural successors, datasets and applications," *Multimedia Tools and Applications*, vol. 82, no. 6, pp. 9243–9275, Mar. 2023. [Online]. Available: https://doi.org/10.1007/s11042-022-13644-y

[2] X. Zhang, T. Zhang, G. Wang, P. Zhu, X. Tang, X. Jia, and L. Jiao, "Remote sensing object detection meets deep learning: A metareview of challenges and advances," *IEEE Geoscience and Remote Sensing Magazine*, vol. 11, no. 4, pp. 8–44, 2023.

[3] P. Dabove, M. Daud, and L. Olivotto, "Revolutionizing urban mapping: deep learning and data fusion strategies for accurate building footprint segmentation," *Scientific Reports*, vol. 14, no. 1, p. 13510, 2024.

[4] "YOLOv8 for RGBD data · Issue #3432 · ultralytics/ultralytics." [Online]. Available: https://github.com/ultralytics/ultralytics/issues/3432